

Lead Call API

GUVI

September 22, 2026

CONTENTS

Lead Call API — reference	
Authentication	
How you use it	
POST /leads/search	
POST /lead/calls	
GET /recording/{callId}	
GET /transcript/{callId}	
GET /health	
Lead filters	
Call filters	
AI query	
Paging	
Rate limits	
Responses you may get	
Lead fields	
Call fields	
Key management	
Worked example	

Lead Call API — reference

Read-only API for lead details, call logs, call recordings and call transcripts.

Base URL: `https://lead-call-api.codingpuppet.com`

Authentication

Every data endpoint takes your API key in a header:

```
X-API-Key: lca_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

Get a key from the admin page at `/admin`, or ask whoever holds the admin token.

How you use it

Leads are found by **filtering**, which gives you `leadId` values; you then ask for that lead's calls, and then for the audio or transcript of a specific call.

```
POST /leads/search    -> leadIds + lead details
POST /lead/calls      -> that lead's call log
GET  /recording/{callId} -> the audio
GET  /transcript/{callId} -> the text
```

If you already hold a `leadId` (or a LeadSquared `externalId`), you can skip straight to `/lead/calls`.

POST `/leads/search`

Find leads and get their details.

Request

```
{
  "filters": {
    "product": ["iit-data-science"],
    "stage": ["New", "Qualified"],
    "createdFrom": 1750000000000
  },
  "sortBy": "created_at",
  "sortDir": "desc",
  "limit": 100,
  "batch": 0,
  "withTotal": false
}
```

Every field is optional. Sending `{}` returns the most recent leads.

Field	Type	Meaning
<code>filters</code>	object	Any combination from Lead filters below. All conditions AND together.
<code>aiQuery</code>	string	A plain-English description of what you want, instead of (or alongside) <code>filters</code> . See AI query .
<code>sortBy</code>	string	<code>created_at</code> (default), <code>updated_at</code> , <code>stage</code> , <code>product</code> , <code>win_probability</code> , <code>last_call_attempted_at</code> , <code>total_no_of_attempts</code> , <code>no_of_calls_connected</code> , <code>conversion_date</code> , <code>first_contact_date</code>
<code>sortDir</code>	string	<code>desc</code> (default) or <code>asc</code>
<code>limit</code>	int	Rows per page, 1–500. Default 100.
<code>batch</code>	int	Which page, counting from <code>0</code> .
<code>withTotal</code>	bool	Ask for an exact total match count.

Response

```
{
  "rows": [
    {
      "leadId": "WN5qOX_yNd1XEVq",
      "externalId": "f7c8195d-5393-11ee-8437-0241145802ba",
      "stage": "New",
      "product": "common",
      "city": null,
      "state": "Karnataka",
      "leadSource": "AI for India",
      "callsConnected": 0,
      "totalAttempts": 0,
      "createdAt": 1694761109000,
      "ownerName": "Rizwan Quadir",
      "ownerEmail": "rizwan@hclguvi.com"
    }
  ],
  "count": 1,
  "limit": 100,
  "batch": 0,
  "hasMore": true,
  "total": null,
  "appliedAiConditions": null,
  "droppedConditions": []
}
```

Field	Meaning
<code>rows</code>	The leads. Full field list under Lead fields .
<code>count</code>	Rows in this response.
<code>limit</code> / <code>batch</code>	Echoed back, as applied.
<code>hasMore</code>	Whether another page exists.
<code>total</code>	Exact match count, when <code>withTotal: true</code> was sent and the CRM supplied one.
<code>appliedAiConditions</code>	The filter your <code>aiQuery</code> resolved to. See AI query .
<code>droppedConditions</code>	Any <code>aiQuery</code> condition that was not applied, with a reason.

POST /lead/calls

One lead's details plus its call log.

Request

```
{
  "leadId": "WN5qOX_8mRhQSh0",
  "filters": {
    "durationMinSec": 30,
    "hasRecording": true
  },
  "sortBy": "created_at",
  "sortDir": "desc",
  "limit": 100,
  "batch": 0
}
```

Field	Type	Meaning
<code>leadId</code>	string	Required. A Superlead <code>leadId</code> or a LeadSquared <code>externalId</code> — both work.
<code>filters</code>	object	Any combination from Call filters below.
<code>aiQuery</code>	string	Plain-English narrowing of the call list.
<code>sortBy</code>	string	<code>created_at</code> (default), <code>updated_at</code> , <code>duration</code> , <code>start_time</code> , <code>pitch_score</code>
<code>sortDir</code>	string	<code>desc</code> (default) or <code>asc</code>
<code>limit</code>	int	1–500. Default 100.
<code>batch</code>	int	Page number from <code>0</code> .
<code>withTotal</code>	bool	Ask for an exact total.

Response

```

{
  "lead": {
    "leadId": "WN5qOX_8mRhQSh0",
    "stage": "Exploring Courses",
    "product": "iit-data-science",
    "callsConnected": 0,
    "ownerName": "Saravana",
    "ownerEmail": "data@hclguvi.com"
  },
  "calls": [
    {
      "callId": "WN5qOX_cNMr8xju",
      "callTime": 1668497590000,
      "startTime": 1668497590000,
      "endTime": 1668497805000,
      "direction": "outbound",
      "durationSec": 215,
      "status": "connected",
      "finalStatus": "completed",
      "pitchScore": null,
      "violations": 0,
      "analysisSummary": null,
      "hasRecording": true,
      "hasTranscript": false,
      "recordingPath": "/recording/WN5qOX_cNMr8xju",
      "transcriptPath": null,
      "ownerName": "Vignesh VR",
      "ownerEmail": "vigneshvr@hclguvi.com"
    }
  ],
  "count": 1,
  "limit": 100,
  "batch": 0,
  "hasMore": false
}

```

`recordingPath` and `transcriptPath` are ready-to-call paths on this API. `hasRecording` and `hasTranscript` tell you which of the two are available for that call — check them before requesting.

GET /recording/{callId}Streams the call audio. Returns `audio/mpeg`.

```
curl -s https://lead-call-api.codingpuppet.com/recording/WN5qOX_rdImIRZi \
-H "X-API-Key: lca_..." -o call.mp3
```

Use it directly as an audio source too:

```
<audio src="https://lead-call-api.codingpuppet.com/recording/{callId}"></audio>
```

(Browsers cannot send the `X-API-Key` header on an `<audio>` tag, so fetch the bytes in your code and hand the player a blob URL.)

GET /transcript/{callId}

The transcript text for a call.

```
curl -s https://lead-call-api.codingpuppet.com/transcript/{callId} -H "X-API-Key: lca_..."
```

```
{
  "callId": "WN5qOX_cNMr8xju",
  "source": "transcript_content",
  "transcript": "Agent: Good morning, am I speaking with...\nLearner: Yes, speaking..."
}
```

`source` tells you where the text came from — useful because the formats differ slightly: `transcript`, `transcript_content` and `transcript_content_vt` come from the CRM's own fields, and `transcript_url` means it was fetched from the CRM's stored transcript file.

GET /health

Liveness check. Needs no API key.

```
{ "status": "ok" }
```

Lead filters

All optional, all AND together. `list` filters mean “any of these”.

Attributes

Filter	Type	Notes
<code>product</code>	list	Matches the lead’s product, covering both product fields the CRM keeps
<code>stage</code>	list	Current funnel stage
<code>previousStage</code>	list	Stage before the current one
<code>language</code>	list	Preferred speaking/learning language
<code>segmentation</code>	list	e.g. <code>M1</code>
<code>city</code> / <code>state</code> / <code>country</code>	list	Location

Source / marketing attribution

Filter	Type
<code>leadSource</code>	list
<code>sourceCampaign</code>	list
<code>sourceMedium</code>	list
<code>sourceContent</code>	list
<code>nurturing</code>	list

Ownership

Filter	Type	Notes
<code>ownerEmail</code>	list	The owning BDA's email
<code>salesOwnerEmail</code>	list	The sales owner's email
<code>team</code>	list	The owner's team name

Call activity (kept on the lead itself, so no need to page the call log)

Filter	Type	Notes
<code>lastDispositionStatus</code>	list	Last call outcome
<code>lastSubDispositionStatus</code>	list	Sub-outcome
<code>attemptsMin</code> / <code>attemptsMax</code>	int	Total call attempts
<code>connectedMin</code> / <code>connectedMax</code>	int	Connected calls
<code>lastCallFrom</code> / <code>lastCallTo</code>	int (ms)	When last attempted

Dates — all epoch milliseconds

Filter	Covers
<code>createdFrom</code> / <code>createdTo</code>	Lead created
<code>updatedFrom</code> / <code>updatedTo</code>	Lead last updated
<code>firstContactFrom</code> / <code>firstContactTo</code>	First contact
<code>conversionFrom</code> / <code>conversionTo</code>	Conversion

Scoring and direct lookup

Filter	Type	Notes
<code>winProbabilityMin</code> / <code>winProbabilityMax</code>	float	Win probability
<code>leadIds</code>	list	Up to 200 Superleap lead ids
<code>externalIds</code>	list	Up to 200 LeadSquared GUIDs

Call filters

Filter	Type	Notes
<code>callFrom</code> / <code>callTo</code>	int (ms)	Call time range
<code>direction</code>	list	e.g. <code>outbound</code>
<code>status</code>	list	e.g. <code>connected</code> , <code>missed_call</code>
<code>finalStatus</code>	list	e.g. <code>completed</code>
<code>durationMinSec</code> / <code>durationMaxSec</code>	int	Call length in seconds
<code>pitchScoreMin</code> / <code>pitchScoreMax</code>	float	Pitch score
<code>hasRecording</code>	bool	Only calls with (or without) audio
<code>hasTranscript</code>	bool	Only calls with (or without) a transcript
<code>ownerEmail</code>	list	The BDA who made the call

AI query

Send `aiQuery` with a plain-English description and the CRM's own query intelligence turns it into filters, grounded in the real stage names and field values.

```
{ "aiQuery": "leads from facebook created in the last 7 days", "limit": 20 }
```

It resolves things a literal filter can't easily express — “last 7 days” becomes a real date range, and shorthand like “DNP” expands to every matching stage name.

The resolved filter comes back in `appliedAiConditions` so you can see exactly what ran:

```
"appliedAiConditions": {
  "and": [{ "and": [{ "field": "lead_source", "operator": "eq", "value":
    "Facebook" }] }]
}
```

Two things worth knowing:

- Combine it with `filters` and both apply — `aiQuery` narrows your explicit filters further.
- For call **duration**, use `filters.durationMinSec` / `durationMaxSec` rather than phrasing it in `aiQuery`; those convert the units correctly.

Anything the query resolves to that wasn't applied is listed in `droppedConditions` with a reason.

Paging

`limit` is rows per page (max 500), `batch` is the page number starting at `0`. `hasMore` tells you to fetch the next one.

```
# page 1
-d '{"filters":{"stage":["New"]},"limit":500,"batch":0}'
# page 2
-d '{"filters":{"stage":["New"]},"limit":500,"batch":1}'
```

Ask for `withTotal: true` when you need the exact match count.

Rate limits

60 requests per 10 seconds per key. Over that you get `429` with a `Retry-After` header telling you how many seconds to wait.

Responses you may get

Code	message	What it means
200	—	Success
401	INVALID_API_KEY	Key missing or not recognised
401	API_KEY_REVOKED	Key was revoked
404	LEAD_NOT_FOUND	No lead with that id
404	CALL_NOT_FOUND	No call with that id
404	RECORDING_NOT_AVAILABLE	That call has no recording
404	TRANSCRIPT_NOT_AVAILABLE	That call has no transcript
400	SORT_FIELD_NOT_ALLOWED	<code>sortBy</code> isn't one of the listed sort fields
400	AI_FILTER_NO_CONDITIONS	The <code>aiQuery</code> phrase didn't describe a filter — rephrase it as a narrowing
400	AI_FILTER_ALL_CONDITIONS_BLOCKED	None of the resolved conditions could be applied
400	AI_FILTER_FAILED	The query service was unreachable — retry
422	VALIDATION_ERROR	A field name or type in your request isn't recognised
429	RATE_LIMITED	Slow down; see <code>Retry-After</code>
502	RECORDING_ACCESS_DENIED	The stored audio file couldn't be fetched from the telephony provider
502	CRM_UNAVAILABLE	The CRM was unreachable — retry

Error bodies look like:

```
{ "status": "error", "message": "LEAD_NOT_FOUND", "errorCode": "E404" }
```

Lead fields

Field	Type
leadId	string
externalId	string — the LeadSquared GUID
stage, previousStage	string
product	string
language	string
winProbability	float
segmentation	string
salesQualified	bool
city, state, country	string
leadSource, sourceCampaign, sourceMedium, sourceContent	string
lastSource, lastMedium	string
nurturing	string
lastDispositionStatus, lastSubDispositionStatus	string
lastCallAttemptedAt	int (ms)
callsConnected, callsMissed, totalAttempts	int
totalTalktimeSec	int (seconds)
firstContactDate, conversionDate	int (ms)
createdAt, updatedAt	int (ms)
ownerId, ownerName, ownerEmail	string
salesOwnerId, salesOwnerName	string

Call fields

Field	Type
<code>callId</code>	string
<code>callTime</code> , <code>startTime</code> , <code>endTime</code>	int (ms)
<code>direction</code>	string
<code>durationSec</code>	int (seconds)
<code>status</code> , <code>finalStatus</code>	string
<code>pitchScore</code>	float
<code>violations</code>	int
<code>analysisSummary</code>	string — the CRM's own short call analysis
<code>hasRecording</code> , <code>hasTranscript</code>	bool
<code>recordingPath</code> , <code>transcriptPath</code>	string — paths on this API
<code>ownerId</code> , <code>ownerName</code> , <code>ownerEmail</code>	string

All timestamps are epoch **milliseconds**.

Key management

The admin page at `/admin` creates and revokes API keys. Open it, enter the admin token, and you can add a key with a tag name or revoke one you no longer need. A new key is shown **once** — copy it then.

The same operations over HTTP, with `X-Admin-Token` instead of `X-API-Key`:

Endpoint	Body	Returns
POST /admin/keys/create	<code>{"name": "who it is for"}</code>	<code>keyId</code> , <code>name</code> , <code>apiKey</code> , <code>keyPrefix</code>
POST /admin/keys/list	<code>{}</code>	Every key with its name, prefix, status, usage count, last-used and created times
POST /admin/keys/revoke	<code>{"keyId": "..."} </code>	<code>{"ok": true}</code>

A revoked key stops working on its next request.

Worked example

Find recent data-science leads, take one, read its calls, pull a transcript.

```
KEY="lca_..."
BASE="https://lead-call-api.codingpuppet.com"

# 1. find leads
curl -s $BASE/leads/search -H "X-API-Key: $KEY" -H "Content-Type: application/json" -d '{
  "filters": { "product": ["iit-data-science"], "createdFrom": 1750000000000 },
  "limit": 50
}'

# 2. that lead's connected calls over 30 seconds
curl -s $BASE/lead/calls -H "X-API-Key: $KEY" -H "Content-Type: application/json" -d '{
  "leadId": "WN5qOX_8mRhQSh0",
  "filters": { "status": ["connected"], "durationMinSec": 30 }
}'

# 3. the transcript of one call
curl -s $BASE/transcript/WN5qOX_cNM8xju -H "X-API-Key: $KEY"

# 4. the audio
curl -s $BASE/recording/WN5qOX_cNM8xju -H "X-API-Key: $KEY" -o call.mp3
```

Python:

```
import requests

BASE = "https://lead-call-api.codingpuppet.com"
HEAD = {"X-API-Key": "lca_..."}

leads = requests.post(f"{BASE}/leads/search", headers=HEAD, json={
    "filters": {"product": ["iit-data-science"], "connectedMin": 1},
    "limit": 200,
}).json()

for lead in leads["rows"]:
    calls = requests.post(f"{BASE}/lead/calls", headers=HEAD, json={
        "leadId": lead["leadId"],
        "filters": {"hasTranscript": True},
    }).json()

    for call in calls["calls"]:
        text = requests.get(f"{BASE}/transcript/{call['callId']}",
headers=HEAD).json()
        print(lead["leadId"], call["callId"], call["durationSec"],
text["transcript"][:200])
```